

```

> restart;
> with(LinearAlgebra);
[&x, Add, Adjoint, BackwardSubstitute, BandMatrix, Basis, BezoutMatrix,      (1)
 BidiagonalForm, BilinearForm, CARE, CharacteristicMatrix,
 CharacteristicPolynomial, Column, ColumnDimension, ColumnOperation,
 ColumnSpace, CompanionMatrix, CompressedSparseForm,
 ConditionNumber, ConstantMatrix, ConstantVector, Copy,
 CreatePermutation, CrossProduct, DARE, DeleteColumn, DeleteRow,
 Determinant, Diagonal, DiagonalMatrix, Dimension, Dimensions,
 DotProduct, EigenConditionNumbers, Eigenvalues, Eigenvectors, Equal,
 ForwardSubstitute, FrobeniusForm, FromCompressedSparseForm,
 FromSplitForm, GaussianElimination, GenerateEquations,
 GenerateMatrix, Generic, GetResultDataType, GetResultShape,
 GivensRotationMatrix, GramSchmidt, HankelMatrix, HermiteForm,
 HermitianTranspose, HessenbergForm, HilbertMatrix,
 HouseholderMatrix, IdentityMatrix, IntersectionBasis, IsDefinite,
 IsOrthogonal, IsSimilar, IsUnitary, JordanBlockMatrix, JordanForm,
 KroneckerProduct, LA_Main, LUDecomposition, LeastSquares,
 LinearSolve, LyapunovSolve, Map, Map2, MatrixAdd, MatrixExponential,
 MatrixFunction, MatrixInverse, MatrixMatrixMultiply, MatrixNorm,
 MatrixPower, MatrixScalarMultiply, MatrixVectorMultiply,
 MinimalPolynomial, Minor, Modular, Multiply, NoUserValue, Norm,
 Normalize, NullSpace, OuterProductMatrix, Permanent, Pivot, PopovForm,
 ProjectionMatrix, QRDecomposition, RandomMatrix, RandomVector, Rank,
 RationalCanonicalForm, ReducedRowEchelonForm, Row, RowDimension,
 RowOperation, RowSpace, ScalarMatrix, ScalarMultiply, ScalarVector,
 SchurForm, SingularValues, SmithForm, SplitForm,
 StronglyConnectedBlocks, SubMatrix, SubVector, SumBasis,
 SylvesterMatrix, SylvesterSolve, ToeplitzMatrix, Trace, Transpose,
 TridiagonalForm, UnitVector, VandermondeMatrix, VectorAdd,
 VectorAngle, VectorMatrixMultiply, VectorNorm, VectorScalarMultiply,
 ZeroMatrix, ZeroVector, Zip]

> # with(linalg):with(Student[LinearAlgebra]): with (LinearAlgebra)
:
> restart;
> with(LinearAlgebra)
[&x, Add, Adjoint, BackwardSubstitute, BandMatrix, Basis, BezoutMatrix,      (2)
 BidiagonalForm, BilinearForm, CARE, CharacteristicMatrix,
 CharacteristicPolynomial, Column, ColumnDimension, ColumnOperation,

```

ColumnSpace, CompanionMatrix, CompressedSparseForm,
 ConditionNumber, ConstantMatrix, ConstantVector, Copy,
 CreatePermutation, CrossProduct, DARE, DeleteColumn, DeleteRow,
 Determinant, Diagonal, DiagonalMatrix, Dimension, Dimensions,
 DotProduct, EigenConditionNumbers, Eigenvalues, Eigenvectors, Equal,
 ForwardSubstitute, FrobeniusForm, FromCompressedSparseForm,
 FromSplitForm, GaussianElimination, GenerateEquations,
 GenerateMatrix, Generic, GetResultDataType, GetResultShape,
 GivensRotationMatrix, GramSchmidt, HankelMatrix, HermiteForm,
 HermitianTranspose, HessenbergForm, HilbertMatrix,
 HouseholderMatrix, IdentityMatrix, IntersectionBasis, IsDefinite,
 IsOrthogonal, IsSimilar, IsUnitary, JordanBlockMatrix, JordanForm,
 KroneckerProduct, LA_Main, LUDecomposition, LeastSquares,
 LinearSolve, LyapunovSolve, Map, Map2, MatrixAdd, MatrixExponential,
 MatrixFunction, MatrixInverse, MatrixMatrixMultiply, MatrixNorm,
 MatrixPower, MatrixScalarMultiply, MatrixVectorMultiply,
 MinimalPolynomial, Minor, Modular, Multiply, NoUserValue, Norm,
 Normalize, NullSpace, OuterProductMatrix, Permanent, Pivot, PopovForm,
 ProjectionMatrix, QRDecomposition, RandomMatrix, RandomVector, Rank,
 RationalCanonicalForm, ReducedRowEchelonForm, Row, RowDimension,
 RowOperation, RowSpace, ScalarMatrix, ScalarMultiply, ScalarVector,
 SchurForm, SingularValues, SmithForm, SplitForm,
 StronglyConnectedBlocks, SubMatrix, SubVector, SumBasis,
 SylvesterMatrix, SylvesterSolve, ToeplitzMatrix, Trace, Transpose,
 TridiagonalForm, UnitVector, VandermondeMatrix, VectorAdd,
 VectorAngle, VectorMatrixMultiply, VectorNorm, VectorScalarMultiply,
 ZeroMatrix, ZeroVector, Zip]

> # Exercise 1

> lambda[1]:=-2

$$\lambda_1 := -2 \quad (3)$$

> lambda[2]:=-1+I;

$$\lambda_2 := -1 + I \quad (4)$$

> lambda[3]:=-1-I;

$$\lambda_3 := -1 - I \quad (5)$$

> u[1]:=Matrix([[0],[0],[1]])

(6)

$$u_1 := \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad (6)$$

> u[2]:=Matrix([[1+I],[1],[0]])

$$u_2 := \begin{bmatrix} 1 + I \\ 1 \\ 0 \end{bmatrix} \quad (7)$$

> u[3]:=Matrix([[1-I],[1],[0]])

$$u_3 := \begin{bmatrix} 1 - I \\ 1 \\ 0 \end{bmatrix} \quad (8)$$

> P:=Matrix([u[1],u[2],u[3]])

$$P := \begin{bmatrix} 0 & 1 + I & 1 - I \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix} \quad (9)$$

> A:=Matrix([[0,-2,0],[1,-2,0],[0,0,-2]]);

$$A := \begin{bmatrix} 0 & -2 & 0 \\ 1 & -2 & 0 \\ 0 & 0 & -2 \end{bmatrix} \quad (10)$$

> D1:=Matrix([[lambda[1],0,0],[0,lambda[2],0],[0,0,lambda[3]]]);

$$D1 := \begin{bmatrix} -2 & 0 & 0 \\ 0 & -1 + I & 0 \\ 0 & 0 & -1 - I \end{bmatrix} \quad (11)$$

> # a) compute those matrices, deduce that uk is an eigenvector corresponding to eigenvalue lambda[k]

> evalm(A &* u[1] - lambda[1]*u[1]);

(12)

$$\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

(12)

```
> evalm(A &* u[1] - lambda[2]*u[1]);
```

$$\begin{bmatrix} 0 \\ 0 \\ -1 - I \end{bmatrix}$$

(13)

```
> evalm(A &* u[2] - lambda[2]*u[2]);
```

$$\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

(14)

```
> evalm(A &* u[2] - lambda[3]*u[2]);
```

$$\begin{bmatrix} -2 + 2I \\ 2I \\ 0 \end{bmatrix}$$

(15)

```
> evalm(A &* u[3] - lambda[3]*u[3]);
```

$$\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

(16)

```
> simplify(evalm(A &* u[1] - lambda[1]*u[1]))
```

$$\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

(17)

```
> simplify(evalm(A &* u[2] - lambda[2]*u[2]))
```

$$\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

(18)

```
> simplify(evalm(A &* u[3] - lambda[3]*u[3]))
```

$$\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

(19)

```
> Eigenvalues(A)
```

$$\begin{bmatrix} -2 \\ -1 - I \\ -1 + I \end{bmatrix}$$

(20)

```
> eigenvalues(A)
```

$$\text{eigenvalues} \left(\begin{bmatrix} 0 & -2 & 0 \\ 1 & -2 & 0 \\ 0 & 0 & -2 \end{bmatrix} \right)$$

(21)

```
> eigenvectors(A)
```

$$\text{eigenvectors} \left(\begin{bmatrix} 0 & -2 & 0 \\ 1 & -2 & 0 \\ 0 & 0 & -2 \end{bmatrix} \right)$$

(22)

```
> Eigenvectors(A)
```

$$\begin{bmatrix} -2 \\ -1 + I \\ -1 - I \end{bmatrix}, \begin{bmatrix} 0 & 1 + I & 1 - I \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

(23)

```
> # because A*u[k]=lambda[k]*u[k], k=1,2,3 => u[k] is an  
eigenvector corresponding to the eigenvalue lambda[k], for each  
k=1,2,3
```

```
> # b) compute a-pdp^-1 and deduce that a is diagonalizable over C
```

```
> B:=evalm(P &* D1 &* P^(-1));
```

$$B := \begin{bmatrix} 0 & -2 & 0 \\ 1 & -2 & 0 \\ 0 & 0 & -2 \end{bmatrix}$$

(24)

```
> B:=LinearAlgebra:-Multiply(P,D1,MatrixInverse(P));
```

Error. (in LinearAlgebra:-Multiply) invalid output option:
 MatrixInverse(Matrix(3, 3, [[0.1+I,1-I],[0.1,1],[1,0,0]]))

> P:=<u[1] | u[2] | u[3]>;

$$P := \begin{bmatrix} 0 & 1+I & 1-I \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix} \quad (25)$$

> B:=LinearAlgebra:-Multiply(P,D1,LinearAlgebra:-MatrixInverse(P));
 Error. (in LinearAlgebra:-Multiply) invalid output option: Matrix(3, 3, [[0,0,1],[-1/2*I,1/2+1/2*I,0],[1/2*I,1/2-1/2*I,0]])

> P:=Matrix([[u[1][1,1],u[2][1,1],u[3][1,1]],[u[1][2,1],u[2][2,1],u[3][2,1]],[u[1][3,1],u[2][3,1],u[3][3,1]]]);

$$P := \begin{bmatrix} 0 & 1+I & 1-I \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix} \quad (26)$$

> P:=Matrix([u[1],u[2],u[3]])

$$P := \begin{bmatrix} 0 & 1+I & 1-I \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix} \quad (27)$$

> u[1]:=Matrix([[0],[0],[1]])

$$u_1 := \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad (28)$$

> u[2]:=Matrix([[1+I],[1],[0]])

$$u_2 := \begin{bmatrix} 1+I \\ 1 \\ 0 \end{bmatrix} \quad (29)$$

> u[3]:=Matrix([[1-I],[1],[0]])

$$u_3 := \begin{bmatrix} 1-I \\ 1 \\ 0 \end{bmatrix} \quad (30)$$

```
> P:=Matrix([u[1],u[2],u[3]])
```

$$P := \begin{bmatrix} 0 & 1+I & 1-I \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix} \quad (31)$$

```
> B:=evalm(P &* D1 &* P^(-1));
```

$$B := \begin{bmatrix} 0 & -2 & 0 \\ 1 & -2 & 0 \\ 0 & 0 & -2 \end{bmatrix} \quad (32)$$

```
> lambda[1]:=-2
```

$$\lambda_1 := -2 \quad (33)$$

```
> lambda[2]:=-1+I;
```

$$\lambda_2 := -1 + I \quad (34)$$

```
> lambda[3]:=-1-I;
```

$$\lambda_3 := -1 - I \quad (35)$$

```
> D1:=Matrix([[lambda[1],0,0],[0,lambda[2],0],[0,0,lambda[3]]]);
```

$$D1 := \begin{bmatrix} -2 & 0 & 0 \\ 0 & -1+I & 0 \\ 0 & 0 & -1-I \end{bmatrix} \quad (36)$$

```
> B:=evalm(P &* D1 &* P^(-1));
```

$$B := \begin{bmatrix} 0 & -2 & 0 \\ 1 & -2 & 0 \\ 0 & 0 & -2 \end{bmatrix} \quad (37)$$

```
> evalm(A-B)
```

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad (38)$$

```
> # c)
```

```
> lambda[1]
```

$$-2 \quad (39)$$

$$\begin{aligned} &> \text{lambda}[2] \\ & \qquad \qquad \qquad -1 + I \end{aligned} \tag{40}$$

$$\begin{aligned} &> \text{lambda}[3] \\ & \qquad \qquad \qquad -1 - I \end{aligned} \tag{41}$$

$$\begin{aligned} &> D1 \\ & \qquad \qquad \qquad \begin{bmatrix} -2 & 0 & 0 \\ 0 & -1 + I & 0 \\ 0 & 0 & -1 - I \end{bmatrix} \end{aligned} \tag{42}$$

$$\begin{aligned} &> \text{LinearAlgebra:-MatrixExponential}(t*D1) \\ & \qquad \qquad \qquad \begin{bmatrix} e^{-2t} & 0 & 0 \\ 0 & e^{-t} \cos(t) + I e^{-t} \sin(t) & 0 \\ 0 & 0 & e^{-t} \cos(t) - I e^{-t} \sin(t) \end{bmatrix} \end{aligned} \tag{43}$$

$$\begin{aligned} &> ee := \text{LinearAlgebra:-MatrixExponential}(t*A); \\ ee := & \begin{bmatrix} e^{-t} \cos(t) + e^{-t} \sin(t) & -2 e^{-t} \sin(t) & 0 \\ e^{-t} \sin(t) & e^{-t} \cos(t) - e^{-t} \sin(t) & 0 \\ 0 & 0 & e^{-2t} \end{bmatrix} \end{aligned} \tag{44}$$

$$\begin{aligned} &> \text{simplify}(ee) \\ & \qquad \qquad \qquad \begin{bmatrix} e^{-t} (\cos(t) + \sin(t)) & -2 e^{-t} \sin(t) & 0 \\ e^{-t} \sin(t) & e^{-t} (\cos(t) - \sin(t)) & 0 \\ 0 & 0 & e^{-2t} \end{bmatrix} \end{aligned} \tag{45}$$

$$\begin{aligned} &> \text{combine}(\%) \\ & \qquad \qquad \qquad \begin{bmatrix} e^{-t} (\cos(t) + \sin(t)) & -2 e^{-t} \sin(t) & 0 \\ e^{-t} \sin(t) & e^{-t} (\cos(t) - \sin(t)) & 0 \\ 0 & 0 & e^{-2t} \end{bmatrix} \end{aligned} \tag{46}$$

> simplify(%)

$$\begin{bmatrix} e^{-t}(\cos(t) + \sin(t)) & -2e^{-t}\sin(t) & 0 \\ e^{-t}\sin(t) & e^{-t}(\cos(t) - \sin(t)) & 0 \\ 0 & 0 & e^{-2t} \end{bmatrix} \quad (47)$$

```
> combine(%%)
```

$$\begin{bmatrix} e^{-t}(\cos(t) + \sin(t)) & -2e^{-t}\sin(t) & 0 \\ e^{-t}\sin(t) & e^{-t}(\cos(t) - \sin(t)) & 0 \\ 0 & 0 & e^{-2t} \end{bmatrix} \quad (48)$$

```
> # d)
```

```
> limit(ee, t=infinity)
```

Error, invalid input: limit expects its 1st argument, expr, to be of type Or(equation,algebraic), but received Matrix(3, 3, [[exp(-t)*cos(t)+exp(-t)*sin(t),-2*exp(-t)*sin(t),0],[exp(-t)*sin(t),exp(-t)*cos(t)-exp(-t)*sin(t),0],[0,0,exp(-2*t)])])

```
> map(x->limit(x,t=infinity),ee)
```

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad (49)$$

```
> for i from 1 to 3 do
  for j from 1 to 3 do
    lim:=limit(ee[i][j],t=infinity);
    print(lim)
  end do:
end do:
```

```
0
0
0
0
0
0
0
0
0
0
```

(50)

```
[> # e)
```

```
> ec1:=diff(x[1](t),t)=-2*x[2](t)
```

$$ec1 := \frac{d}{dt} x_1(t) = -2 x_2(t) \quad (51)$$

```
> ec2:=diff(x[2](t),t)=x[1](t)-2*x[2](t)
```

$$ec2 := \frac{d}{dt} x_2(t) = x_1(t) - 2 x_2(t) \quad (52)$$

```
> ec3:=diff(x[3](t),t)=-2*x[3](t)
```

$$ec3 := \frac{d}{dt} x_3(t) = -2 x_3(t) \quad (53)$$

```
> sist:=ec1,ec2,ec3;
```

$$sist := \frac{d}{dt} x_1(t) = -2 x_2(t), \frac{d}{dt} x_2(t) = x_1(t) - 2 x_2(t), \frac{d}{dt} x_3(t) = -2 x_3(t) \quad (54)$$

```
> sol:=dsolve({sist},{x[1],x[2],x[3]});
```

$$sol := \left\{ x_1(t) = e^{-t} (\sin(t) c_1 + \cos(t) c_2), x_2(t) = \frac{e^{-t} (\sin(t) c_1 + \sin(t) c_2 - \cos(t) c_1 + \cos(t) c_2)}{2}, x_3(t) = c_3 e^{-2t} \right\} \quad (55)$$

```
> rhs(sol[1]);
```

$$e^{-t} (\sin(t) c_1 + \cos(t) c_2) \quad (56)$$

```
> limit(rhs(sol[1]),t=infinity);
```

$$0 \quad (57)$$

```
> limit(rhs(sol[2]),t=infinity);
```

$$0 \quad (58)$$

```
> limit(rhs(sol[3]),t=infinity);
```

$$0 \quad (59)$$

```
> # Exercise 2
```

```
> restart;
```

```
> with(LinearAlgebra)
```

[&x, Add, Adjoint, BackwardSubstitute, BandMatrix, Basis, BezoutMatrix, BidiagonalForm, BilinearForm, CARE, CharacteristicMatrix, CharacteristicPolynomial, Column, ColumnDimension, ColumnOperation, ColumnSpace, CompanionMatrix, CompressedSparseForm, ConditionNumber, ConstantMatrix, ConstantVector, Copy, CreatePermutation, CrossProduct, DARE, DeleteColumn, DeleteRow, Determinant, Diagonal, DiagonalMatrix, Dimension, Dimensions, DotProduct, EigenConditionNumbers, Eigenvalues, Eigenvectors, Equal, ForwardSubstitute, FrobeniusForm, FromCompressedSparseForm, (60)

FromSplitForm, GaussianElimination, GenerateEquations, GenerateMatrix, Generic, GetResultDataType, GetResultShape, GivensRotationMatrix, GramSchmidt, HankelMatrix, HermiteForm, HermitianTranspose, HessenbergForm, HilbertMatrix, HouseholderMatrix, IdentityMatrix, IntersectionBasis, IsDefinite, IsOrthogonal, IsSimilar, IsUnitary, JordanBlockMatrix, JordanForm, KroneckerProduct, LA_Main, LUDecomposition, LeastSquares, LinearSolve, LyapunovSolve, Map, Map2, MatrixAdd, MatrixExponential, MatrixFunction, MatrixInverse, MatrixMatrixMultiply, MatrixNorm, MatrixPower, MatrixScalarMultiply, MatrixVectorMultiply, MinimalPolynomial, Minor, Modular, Multiply, NoUserValue, Norm, Normalize, NullSpace, OuterProductMatrix, Permanent, Pivot, PopovForm, ProjectionMatrix, QRDecomposition, RandomMatrix, RandomVector, Rank, RationalCanonicalForm, ReducedRowEchelonForm, Row, RowDimension, RowOperation, RowSpace, ScalarMatrix, ScalarMultiply, ScalarVector, SchurForm, SingularValues, SmithForm, SplitForm, StronglyConnectedBlocks, SubMatrix, SubVector, SumBasis, SylvesterMatrix, SylvesterSolve, ToeplitzMatrix, Trace, Transpose, TridiagonalForm, UnitVector, VandermondeMatrix, VectorAdd, VectorAngle, VectorMatrixMultiply, VectorNorm, VectorScalarMultiply, ZeroMatrix, ZeroVector, Zip]

> D1:=LinearAlgebra:-DiagonalMatrix([2,2,-1,0]);

$$D1 := \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (61)$$

> P:=LinearAlgebra:-RandomMatrix(4,generator=-1..1);

$$P := \begin{bmatrix} -1 & 0 & 1 & 1 \\ -1 & 0 & -1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} \quad (62)$$

> A:=simplify(P.D1.P^(-1));

$$A := \begin{bmatrix} \frac{1}{2} & -\frac{1}{2} & 0 & -2 \\ \frac{3}{2} & -\frac{3}{2} & 0 & -2 \\ -\frac{3}{2} & -\frac{1}{2} & 2 & -2 \\ -\frac{3}{2} & \frac{3}{2} & 0 & 2 \end{bmatrix} \quad (63)$$

> eigenvals(A)

$$\text{eigenvals} \left(\begin{bmatrix} \frac{1}{2} & -\frac{1}{2} & 0 & -2 \\ \frac{3}{2} & -\frac{3}{2} & 0 & -2 \\ -\frac{3}{2} & -\frac{1}{2} & 2 & -2 \\ -\frac{3}{2} & \frac{3}{2} & 0 & 2 \end{bmatrix} \right) \quad (64)$$

> eigenvalues(A)

$$\text{eigenvalues} \left(\begin{bmatrix} \frac{1}{2} & -\frac{1}{2} & 0 & -2 \\ \frac{3}{2} & -\frac{3}{2} & 0 & -2 \\ -\frac{3}{2} & -\frac{1}{2} & 2 & -2 \\ -\frac{3}{2} & \frac{3}{2} & 0 & 2 \end{bmatrix} \right) \quad (65)$$

> Eigenvalues(A);

(66)

$$\begin{bmatrix} 0 \\ -1 \\ 2 \\ 2 \end{bmatrix}$$

(66)

> Eigenvectors(A)

$$\begin{bmatrix} 0 \\ -1 \\ 2 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 & 1 & -1 & 0 \\ 1 & -1 & -1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$

(67)

> # b)

> Determinant(A)

0

(68)

> CharacteristicPolynomial(A,lambda)

$$\lambda^4 - 3\lambda^3 + 4\lambda$$

(69)

> solve(lambda^4 - 3*lambda^3 + 4*lambda=0,lambda);

0, -1, 2, 2

(70)

> Eigenvectors(A);

$$\begin{bmatrix} 0 \\ 2 \\ 2 \\ -1 \end{bmatrix}, \begin{bmatrix} 1 & 0 & -1 & 1 \\ 1 & 0 & -1 & -1 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

(71)

> JordanForm(A);

$$\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 2 \end{bmatrix}$$

(72)

> MatrixInverse(A);

Error. (in MatrixInverse) singular matrix

> # METHOD GAUSS-JORDAN: zeroes both under and above main diagonal
- used for big stuff

> # Exercise 3

```
> A:=Matrix([[2,1,0],[0,2,1],[0,0,2]]);
```

$$A := \begin{bmatrix} 2 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{bmatrix}$$

(73)

```
> B:=Matrix([[0,1,0,0],[0,0,1,0],[0,0,0,1],[-1,0,-2,0]])
```

$$B := \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -1 & 0 & -2 & 0 \end{bmatrix}$$

(74)

```
> # a) find eigenvectors and Jordan form. A and B diagonalizable?
```

```
> with(LinearAlgebra):
```

```
> Eigenvalues(A)
```

$$\begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix}$$

(75)

```
> Eigenvectors(A)
```

$$\begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

(76)

```
> Eigenvalues(B)
```

$$\begin{bmatrix} I \\ -I \\ I \\ -I \end{bmatrix}$$

(77)

```
> Eigenvectors(B)
```

(78)

$$\begin{bmatrix} I \\ I \\ -I \\ -I \end{bmatrix}, \begin{bmatrix} I & 0 & -I & 0 \\ -1 & 0 & -1 & 0 \\ -I & 0 & I & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \quad (78)$$

> JordanForm(A)

$$\begin{bmatrix} 2 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{bmatrix} \quad (79)$$

> JordanForm(B)

$$\begin{bmatrix} -I & 1 & 0 & 0 \\ 0 & -I & 0 & 0 \\ 0 & 0 & I & 1 \\ 0 & 0 & 0 & I \end{bmatrix} \quad (80)$$

> P:=RandomMatrix(4,generator=-1..1)

$$P := \begin{bmatrix} 1 & 1 & 0 & -1 \\ -1 & 1 & -1 & 0 \\ 1 & 1 & -1 & 0 \\ 1 & 0 & -1 & 0 \end{bmatrix} \quad (81)$$

> evalm(P &* D1 &* P^(-1));

$$\begin{bmatrix} 0 & -1 & 3 & -2 \\ 0 & \frac{1}{2} & \frac{3}{2} & -3 \\ 0 & -\frac{3}{2} & \frac{7}{2} & -3 \\ 0 & -\frac{3}{2} & \frac{3}{2} & -1 \end{bmatrix} \quad (82)$$

> # b) e^(tA), e^(tB)

> MatrixExponential(t*A)

$$\begin{bmatrix} e^{2t} & te^{2t} & \frac{t^2 e^{2t}}{2} \\ 0 & e^{2t} & te^{2t} \\ 0 & 0 & e^{2t} \end{bmatrix} \quad (83)$$

> MatrixExponential(t*B)

$$\begin{bmatrix} \cos(t) + \frac{\sin(t)t}{2} & -\frac{t\cos(t)}{2} + \frac{3\sin(t)}{2} & \frac{t^2\cos(t)}{2} - \frac{3t\sin(t)}{2} \\ \frac{t\cos(t)}{2} - \frac{\sin(t)}{2} & \cos(t) + \frac{\sin(t)t}{2} & \frac{t\cos(t)}{2} - \frac{3\sin(t)}{2} \\ -\frac{\sin(t)t}{2} & \frac{t\cos(t)}{2} - \frac{\sin(t)}{2} & \cos(t) + \frac{\sin(t)t}{2} \\ -\frac{t\cos(t)}{2} - \frac{\sin(t)}{2} & -\frac{\sin(t)t}{2} & -\frac{t\cos(t)}{2} + \frac{3\sin(t)}{2} \end{bmatrix} \quad (84)$$

> # I DON't really understand the output bc why do we have t*e^2t and not just e^2t, we take each entry of the matrix and we do exponential of it, are the resulting entries solutions or what??

> Determinant(A)

$$8 \quad (85)$$

> D1:=Matrix([[2,0,0],[0,2,0],[0,0,2]]);

$$D1 := \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix} \quad (86)$$

> P:=RandomMatrix(3,generator=-1..1)

$$P := \begin{bmatrix} 1 & 1 & -1 \\ 1 & 0 & 1 \\ 0 & -1 & 1 \end{bmatrix} \quad (87)$$

> A:=simplify(P.D1.P^(-1));

$$(88)$$

$$A := \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{bmatrix} \quad (88)$$

```
> Dimension(D1)
3, 3 (89)
```

```
> Dimension(D1)
3, 3 (90)
```

```
> Dimension(P)
3, 3 (91)
```

```
> D2:=Matrix([[1,0,0,0],[0,-1,0,0],[0,0,1,0],[0,0,0,-1]])
```

$$D2 := \begin{bmatrix} I & 0 & 0 & 0 \\ 0 & -I & 0 & 0 \\ 0 & 0 & I & 0 \\ 0 & 0 & 0 & -I \end{bmatrix} \quad (92)$$

```
> P1:=RandomMatrix(4,generator=-1..1)
```

$$P1 := \begin{bmatrix} -1 & 1 & -1 & 0 \\ -1 & 1 & -1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & -1 & 0 & -1 \end{bmatrix} \quad (93)$$

```
> B:=simplify(P1.D2.P1(-1))
```

$$B := \begin{bmatrix} I & I & I & 0 \\ I & I & I & 0 \\ -I & 0 & -I & I \\ -I & -I & 0 & -I \end{bmatrix} \quad (94)$$

```
> # conclusion: A seems diagonalizable, B does not.
```

```
> # Exercise 4
```

```
> # a)
```

```
> with(DEtools);
```

```
[AreSimilar, Closure, DENormal, DEplot, DEplot3d, DEplot_polygon,
DFactor, DFactorLCLM, DFactorsols, Dchangevar, Desingularize,
FindODE, FunctionDecomposition, GCRD, Gosper, Heunsols, (95)
```

Homomorphisms, IVPsol, IsHyperexponential, LCLM, MeijerGsols, MultiplicativeDecomposition, ODEInvariants, PDEchangecoords, PolynomialNormalForm, RationalCanonicalForm, ReduceHyperexp, RiemannPsols, Xchange, Xcommutator, Xgauge, Zeilberger, abelsol, adjoint, autonomous, bernoullisol, buildsol, buildsym, canoni, caseplot, casesplit, checkrank, chinisol, clairautsol, constcoeffsols, convertAlg, convertsys, dalembertsol, dcoeffs, de2diffop, dfieldplot, diff_table, diffop2de, dperiodic_sols, dpolyform, dsubs, eigenring, endomorphism_charpoly, equinv, eta_k, eulersols, exactsol, expsols, exterior_power, firint, firtest, formal_sol, gen_exp, generate_ic, genhomosol, gensys, hamilton_eqs, hypergeometricsols, hypergeomsols, hyperode, indicialeq, infgen, initialdata, integrate_sols, intfactor, invariants, kovacicols, leftdivision, liesol, line_int, linearsol, matrixDE, matrix_riccati, maxdimsystems, moser_reduce, muchange, mult, mutest, newton_polygon, normalG2, ode_int_y, ode_y1, odeadvisor, odepde, parametricsol, particularsol, phaseportrait, poincare, polysols, power_equivalent, rational_equivalent, ratsols, redode, reduceOrder, reduce_order, regular_parts, regularsp, remove_RootOf, riccati_system, riccatisol, rifread, rifsimp, rightdivision, rtaylor, separablesol, singularities, solve_group, super_reduce, symgen, symmetric_power, symmetric_product, symtest, transinv, translate, untranslate, varparam, zoom]

> with(plots)

[animate, animate3d, animatecurve, arrow, changecoords, complexplot, complexplot3d, conformal, conformal3d, contourplot, contourplot3d, coordplot, coordplot3d, densityplot, display, dualaxisplot, fieldplot, fieldplot3d, gradplot, gradplot3d, implicitplot, implicitplot3d, inequal, interactive, interactiveparams, intersectplot, listcontplot, listcontplot3d, listdensityplot, listplot, listplot3d, loglogplot, logplot, matrixplot, multiple, odeplot, pareto, plotcompare, pointplot, pointplot3d, polarplot, polygonplot, polygonplot3d, polyhedra_supported, polyhedraplot, rootlocus, semilogplot, setcolors, setoptions, setoptions3d, shadebetween, spacecurve, sparsematrixplot, surfdata, textplot, textplot3d, tubeplot]

(96)

> eq:=diff(x(t),t)=1-x(t)^2

$$eq := \frac{d}{dt} x(t) = 1 - x(t)^2$$

(97)

> # a) solve the equation 1-x^2=0, find fi(t,-1) and fi(t,1), plot

graphs, write images denoted by gamma -1, gamma 1

```
> solve(1-x^2=0,x)
```

$$-1, 1 \quad (98)$$

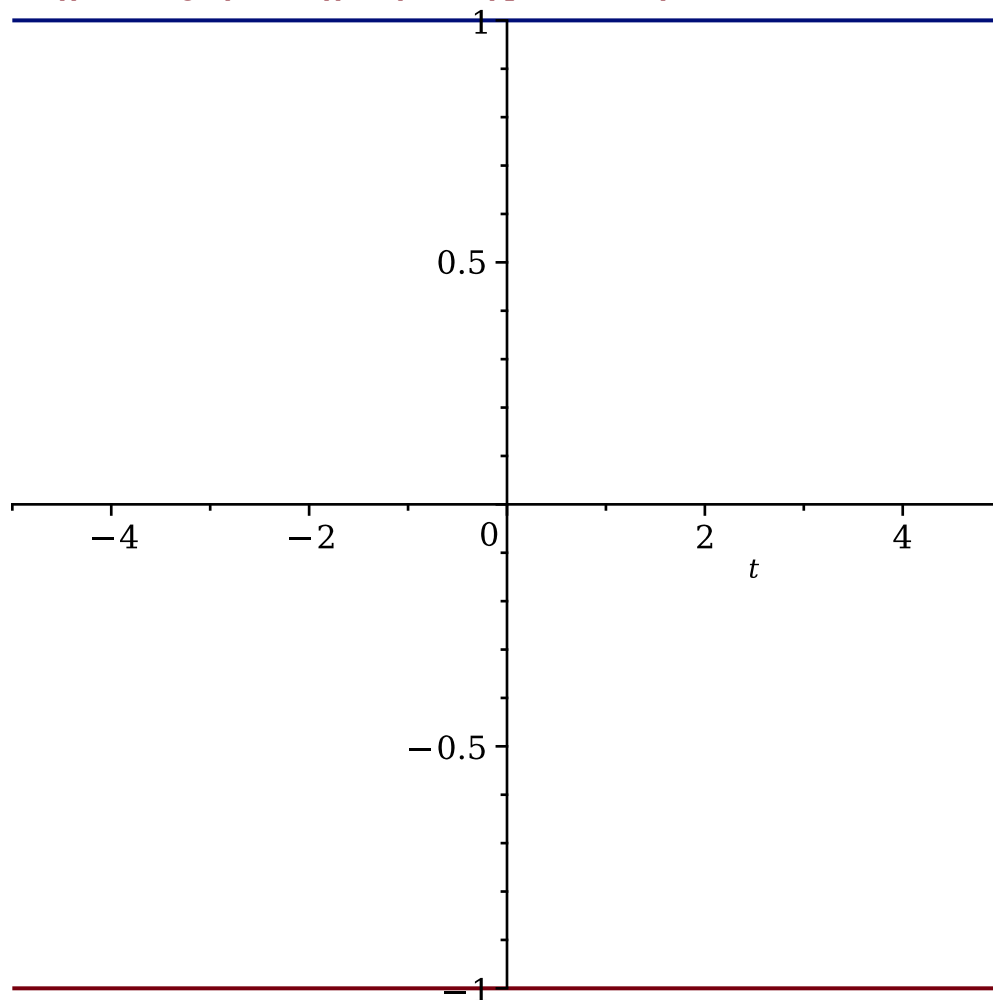
```
> phineg1:=dsolve({eq,x(0)=-1},x(t))
```

$$phineg1 := x(t) = -1 \quad (99)$$

```
> phipos1:=dsolve({eq,x(0)=1},x(t))
```

$$phipos1 := x(t) = 1 \quad (100)$$

```
> plot([rhs(phineg1),rhs(phipos1)],t=-5..5)
```



```
> # b) find fi(t,0), obtain the expression given, find limit at  
infinity and -infinity, plot graphs on R, check if this function  
is strictly increasing, find image denoted by gamma0
```

```
> sol0:=dsolve({eq,x(0)=0},x(t))
```

$$sol0 := x(t) = \tanh(t) \quad (101)$$

```
> phi0:=convert(convert(rhs(sol0),exp),exp)
```

$$\phi0 := \frac{e^{2t} - 1}{e^{2t} + 1} \quad (102)$$

```
> limit(phi0,t=infinity)
```

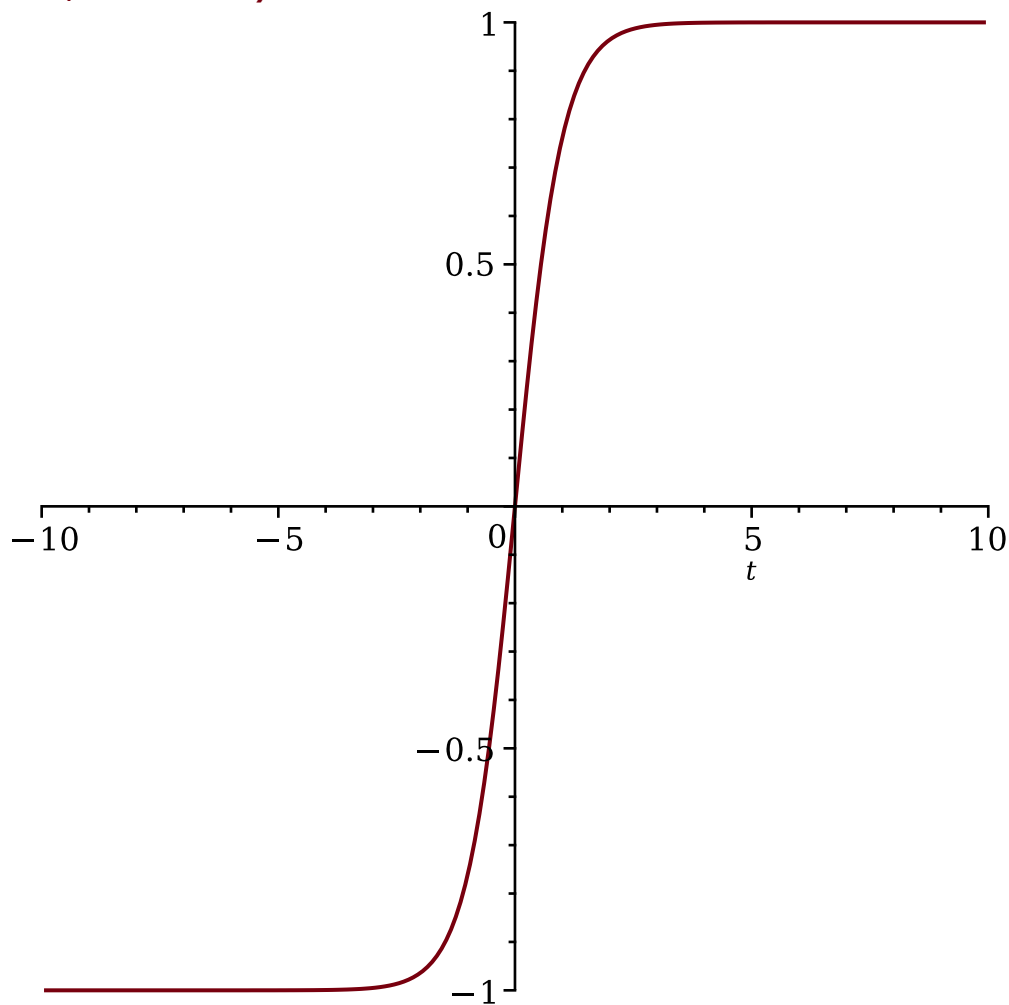
$$1 \quad (103)$$

```
> limit(phi0,t=-infinity)
```

— 1

(104)

```
> plot(phi0,t=-10..10)
```



```
> # c) find fi(t,-2)
```

```
> solneg2:=dsolve({eq,x(0)=-2},x(t))
```

$$solneg2 := x(t) = \coth\left(-\operatorname{arctanh}\left(\frac{1}{2}\right) + t\right)$$

(105)

```
> phineg2:=convert(convert(rhs(solneg2),exp),exp)
```

$$phineg2 := \frac{e^{2t} + 3}{e^{2t} - 3}$$

(106)

```
> limit(phineg2,t=-infinity)
```

— 1

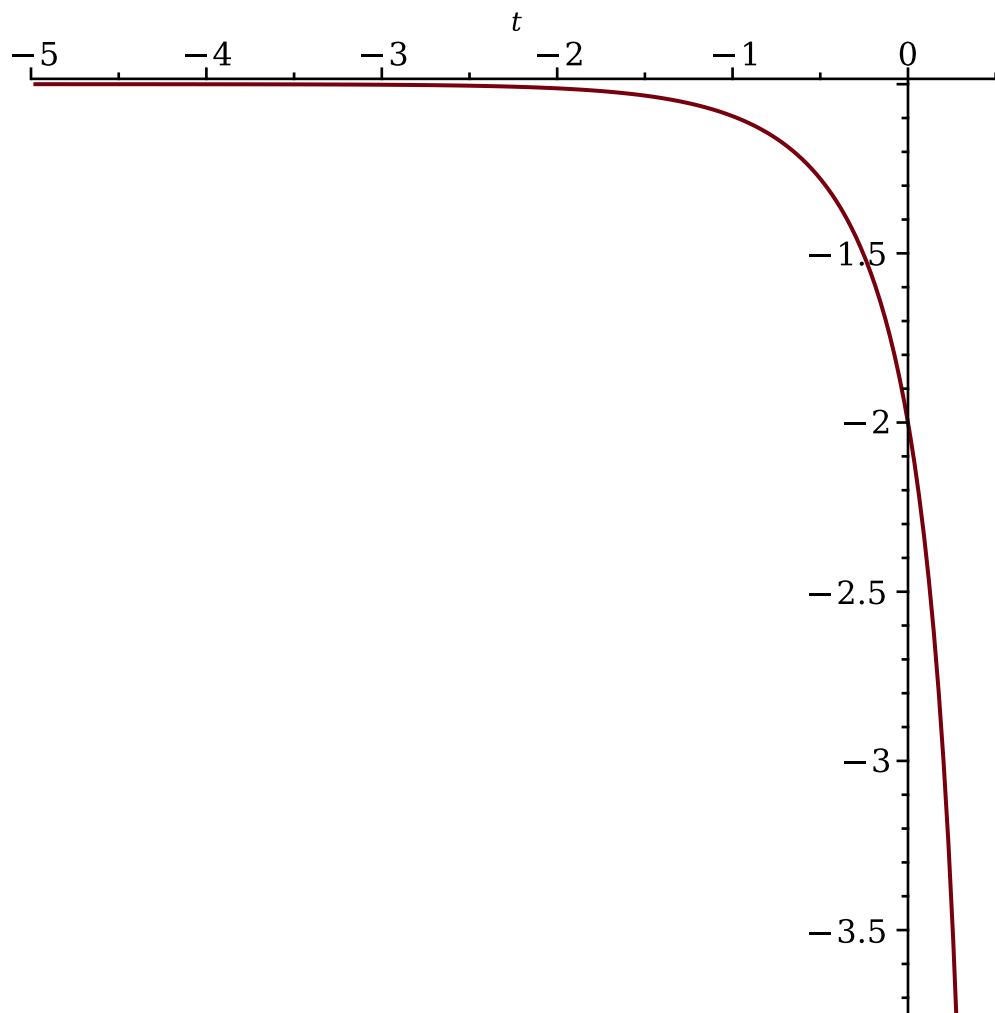
(107)

```
> limit(phineg2,t=ln(sqrt(3)),left)
```

— ∞

(108)

```
> plot(phineg2, t=-5..ln(sqrt(3)))
```



```
> # d) fi(t,2)
> solpos2:=dsolve({eq,x(0)=2},x(t))
      solpos2 := x(t) = coth( arctanh( 1/2 ) + t )
```

(109)

```
> phipos2:=convert(convert(rhs(solpos2),exp),exp)
      phipos2 := (3 e^{2 t} + 1) / (3 e^{2 t} - 1)
```

(110)

```
> limit(phipos2,t=-ln(sqrt(3)),right)
      ∞
```

(111)

```
> limit(phipos2,t=infinity)
      1
```

(112)

```
> plot(phipos2,t=-ln(sqrt(3))..5)
```

